VibeShift: Big Data Analytics for Acoustic Vibe-Shift Discovery in Popular Music

BDA mini project submitted in partial fulfillment of the requirements of the degree of

B. E. Computer Engineering

By

Name	Roll No	PID No
Aryan Shah	30	232155
Saish Sawant	25	232148
Nikhil Sharma	33	232157

Name of the Guide: Prof. _____

Designation: Assistant Professor

Department of Computer Engineering

St. Francis Institute of Technology

(Engineering College)

An Autonomous Institute, Affiliated to University of Mumbai

2025-2026

CERTIFICATE

This is to certify that the mini project entitled “VibeShift: Big Data Analytics for Acoustic Vibe-Shift Discovery in Popular Music” is a bonafide work of Aryan Shah (Roll No. 30, PID 232155), Saish Sawant (Roll No. 25, PID 232148), Nikhil Sharma (Roll No. 33, PID 232157), submitted to the University of Mumbai in partial fulfillment of the requirement for the Big Data Analytics (CSC702) subject in the final year of Computer Engineering. The work was carried out under my supervision and has not been submitted to any other university for the award of any degree or diploma.

Ms. _____ Guide

DECLARATION

We declare that this written submission represents our ideas in our own words and that where others' ideas or words have been included, we have adequately cited and referenced the original sources. We further declare that we have adhered to all principles of academic honesty and integrity and have not misrepresented, fabricated or falsified any idea, data, fact or source in this submission.

_____ Aryan Shah (Roll No. 30)

_____ Saish Sawant (Roll No. 25)

_____ Nikhil Sharma (Roll No. 33)

Date:

ABSTRACT

VibeShift is a full-stack big data analytics application that quantifies how the acoustic character of popular music has shifted across release decades. The corpus is the public Spotify “1.2M+ Songs” dataset: 1,204,025 tracks described by 24 attributes, of which eight audio features form the analysis vector. Rows are cleaned, standardised, clustered with K-Means and projected with PCA; the per-track assignments are aggregated into a decade × cluster matrix that exposes the vibe shift, and each cluster is named from its own measured profile. A background runner executes fifteen instrumented stages per run and streams their timings to the browser. Beyond the clustering pipeline the project implements a Big Data Analytics suite mapped to the CSC702 laboratory manual: an in-process MapReduce runtime with per-phase counters, MongoDB persistence of schemaless run documents, statistical profiling and visualization, an HDFS placement simulation, a Bloom-filter membership index, similarity-graph mining, streaming micro-batches and supervised model selection. The core portion of this report is MapReduce (Experiments 4 and 5), MongoDB/NoSQL (Experiment 3) and statistics with visualization (Experiment 8); the remaining analyses run on every execution and are reported as supporting evidence. A full-corpus run finishes in 26 seconds on one machine and reaches a silhouette score of 0.192 for five clusters.

CONTENTS

Chapter / Section	Title	Page
1	Introduction	1
1.1	Description	1
1.2	Problem Formulation	1
1.3	Motivation	1
1.4	Proposed Solution	2
1.5	Scope of the Project	2
2	Review of Literature	2
3	System Analysis	3
3.1	Functional Requirements	3
3.2	Non-Functional Requirements	3
	Performance Requirements	3
	Software Quality Attributes	4
3.3	Specific Requirements	4
3.4	Use-Case Diagram and Description	4
4	Analysis Modeling	5
4.1	Class Diagram and Activity Diagram	5
4.2	Functional Modelling	6
5	Design	7
5.1	Architectural Design	7
5.2	User Interface Design	8
6	Implementation	9
6.1	Working of the Project	9
6.2	MapReduce – Core Portion (Experiments 4 and 5)	10
6.3	NoSQL Persistence with MongoDB – Core Portion (Experiment 3)	10
6.4	Statistical Computing and Visualization – Core Portion (Experiment 8)	11
6.5	Additional Big Data Analytics Analyses	11
6.6	Concept Catalogue and Track Explorer	12
7	Conclusion	13
References		14
Acknowledgements		14

LIST OF FIGURES

Fig. No.	Figure Caption	Page
3.1	Use-case diagram of VibeShift	5
4.1	Class diagram of the backend components	6
4.2	Data-flow diagram: context (level 0) and decomposition (level 1)	6
5.1	Overall system architecture of VibeShift	7
5.2	Dashboard: corpus profile, run configuration, live console and run history	8
5.3	Results page: headline metrics, vibe-shift heatmap, cluster profiles and projection	9
6.1	MapReduce execution model with the counters recorded for the word-count job	10
6.2	Statistics and visualization: correlation matrix and cluster/decade composition	11
6.3	Big Data Analytics suite: MapReduce inspector, NoSQL console, statistics panel and additional analyses	12

LIST OF TABLES

Table No.	Table Caption	Page
2.1	Comparison of related approaches with VibeShift	3
3.1	Functional requirements	3
3.2	Hardware and software requirements	4
5.1	Technology stack	7
6.1	Description of the corpus used for the recorded run	9
6.2	Measured pipeline cost and headline metrics for the recorded run	9
6.3	Big Data Analytics analyses executed in the recorded run	12

LIST OF ABBREVIATIONS

Sr. No.	Abbreviation	Expanded form
1	API	Application Programming Interface
2	BDA	Big Data Analytics
3	DFD	Data Flow Diagram
4	HDFS	Hadoop Distributed File System
5	k-NN	k-Nearest Neighbours
6	NoSQL	Not Only SQL
7	PCA	Principal Component Analysis
8	REST	Representational State Transfer
9	SPA	Single Page Application

Chapter 1: Introduction

1.1 Description

Popular music changes character over time: recordings from the 1960s are dominated by acoustic, low-energy material, while the 2010s are dominated by loud, electronic, high-energy productions. Streaming platforms describe every track with a numeric audio feature vector, which makes that change measurable at corpus scale. VibeShift ingests a public Spotify corpus, clusters its tracks by acoustic similarity and presents the resulting decade-by-decade “vibe shift” as interactive dashboards. It is a three-tier system: a React single page application, a FastAPI service that executes the pipeline on a background worker, and a persistence layer that keeps every run as a MongoDB document beside columnar Parquet files. Alongside the clustering pipeline it implements the techniques of the CSC702 laboratory manual – distributed storage, MapReduce, NoSQL, probabilistic data structures, graph mining, stream processing, supervised learning and visualization – and shows the live evidence each of them produces.

1.2 Problem Formulation

Given a corpus of tracks $D = \{(x_i, y_i)\}$, where x_i is the feature vector of track i and y_i its release year, the task is to partition the corpus into k acoustically coherent groups, express the composition of each release decade as a distribution over those groups, and present the movement of those distributions over time with the evidence of every technique used to obtain it. The clustering step therefore minimises the within-cluster sum of squares over k centroids after standardisation, and the aggregation step computes $P(\text{decade} \mid \text{cluster})$ as a percentage matrix. Three practical difficulties must be handled: the corpus is large enough that a naive implementation is wasteful, it is dirty (malformed years, duplicate identifiers, quoted fields containing commas), and the clusters K-Means returns are unlabelled integers whose meaning has to be derived before anything can be reported.

1.3 Motivation

The project began as a Colab notebook that ran the same pipeline interactively. The analytics were correct, but every parameter change required re-running cells, no result was persisted and none of the distributed techniques of the manual were exercised. The motivation was to turn that notebook into a system whose runs are configurable, persisted and auditable, and to make the analytics themselves the subject of the application.

1.4 Proposed Solution

A run is submitted from the dashboard and executed by a background worker in fifteen stages: ingest, clean, assemble, K-Means, PCA, aggregate, benchmark and eight Big Data Analytics analyses. Each stage writes progress, counters and timings into the run document, so the browser can show a live console and the results page can justify every number it displays. The pipeline is engine-agnostic: a PySpark MLlib implementation is used when Spark is importable and a single-node pandas and scikit-learn implementation with stage-for-stage parity otherwise, with the active engine and the reason for the choice shown on the dashboard.

1.5 Scope of the Project

In line with the selected scope, MapReduce (Experiments 4 and 5), NoSQL persistence with MongoDB (Experiment 3) and statistical computing with visualization (Experiment 8) form the core portion of the project and receive the detailed treatment in this report. The remaining analyses – HDFS simulation, Bloom filter, similarity-graph mining, streaming windows and supervised learning – stay implemented, run on every execution and are reported as supporting evidence. A single-node deployment, the public corpus, reproducible notebook export and the web interface are in scope; a multi-node cluster, live streaming from a production broker, user accounts and paid infrastructure are out of scope.

Chapter 2: Review of Literature

The project draws on three literatures: the programming models used to process large datasets, the unsupervised learning used to summarise them, and the visualization that makes the summary readable. Dean and Ghemawat introduced MapReduce, showing that a restricted map/shuffle/reduce interface can be parallelised automatically while retaining fault tolerance [1]; Hadoop packaged it with the Hadoop Distributed File System, which splits files into fixed-size blocks and replicates them rack-aware across DataNodes [2]; Spark kept intermediate data in memory and exposed MLlib, whose K-Means and PCA implementations underlie the pipeline used here [3]. For the analytical core, MacQueen's k-means formulation [4] and the treatment of principal component analysis in Hastie et al. [5] provide the basis, while the silhouette criterion reported for cluster quality comes from Rousseeuw [6]. Similarity analysis is more recent: Newman's modularity [7] underlies the community detection and PageRank [8] supplies the centrality measure; Bloom's probabilistic structure

[9] indexes the corpus vocabulary, and the event-time windowing model follows the Dataflow abstraction of Akidau et al. [10].

On the storage side the MongoDB manual documents the document model, reflection, queries and the aggregation pipeline [11], and Kleppmann [12] analyses the trade-offs that motivate keeping schemaless run records beside columnar track files. Commercial services such as the Spotify audio-features API [13] expose per-track descriptors but leave cross-decade aggregation to the consumer. Table 2.1 compares the available approaches with VibeShift.

Table 2.1: Comparison of related approaches with VibeShift

Approach	Processing model	Cross-decade analysis	Evidence of techniques
Colab notebook (baseline)	Single-node, manual cells	Ad hoc tables and plots	None – results are not persisted
Spark MLlib clustering	Distributed K-Means / PCA	Requires custom aggregation	Cluster metrics only
Spotify audio-features API	Hosted per-track lookup	Not provided	Not applicable
VibeShift (this project)	Engine-agnostic 15-stage runner	Decade × cluster shares	Per-technique counters in the UI

Chapter 3: System Analysis

3.1 Functional Requirements

The functional requirements follow the workflow of the analyst who uses the application: inspect the corpus, configure a run, watch it execute, study the results and export the evidence. Table 3.1 lists them with the interface element or service that satisfies each one.

Table 3.1: Functional requirements

ID	Requirement	Satisfied by
FR1	Detect the corpus and report its size, row/column count and real year range	Dataset status API
FR2	Configure k, the year window (bounded by the dataset), features, best-k sweep and sampling	Dashboard run configuration
FR3	Execute the pipeline asynchronously and stream per-stage status, log and timings	Runner thread and run status API
FR4	Cluster the corpus, project it with PCA and aggregate decade × cluster shares	clean / assemble / kmeans / pca / aggregate stages
FR5	Execute the Big Data Analytics suite and record a counter for every analysis	Eight analytics stages
FR6	Map each concept to its experiment and show the live metric it produced	Concepts API and page
FR7	Browse and filter tracks, persist runs, export a Colab notebook	Track store, run store, notebook route

3.2 Non-Functional Requirements

3.2.1 Performance Requirements

A full-corpus run must finish within two minutes on one machine; the recorded run over 1,204,025 rows took 25.6 seconds, of which 4.7 seconds were K-Means and 10.2 seconds the

eight analyses. The API must never block while a run executes, and quadratic work must be bounded: silhouette scoring uses a random sample of 8,000 rows and the analytics use capped samples whose sizes are reported in the interface. Vectorised aggregation is benchmarked against a naive row loop on every run, which measured a 12.8× speed-up here.

- Responsiveness: status polling stays well under a second while a run executes.
- Reliability: a failing analytics stage must not fail the run.
- Portability: the pipeline runs with or without PySpark on the same instruction set.
- Reproducibility: fixed seeds and an exportable notebook for every run.

3.2.2 Software Quality Attributes

Usability is addressed by a task-oriented interface with a live console, so a long analysis never looks stalled. Auditability is treated as a first-class attribute: every displayed number is traceable to a stage counter and the concepts page names the JSON path holding each metric. Correctness is protected by tests of the cleaning rules, the engine contract, the CSV reader, the label derivation and the API lifecycle.

3.3 Specific Requirements

Table 3.2: Hardware and software requirements

Category	Requirement
Processor and memory	x86-64 dual-core minimum, quad-core recommended; 8 GB RAM minimum, 16 GB recommended (the 330 MB CSV expands to about 2 GB in memory)
Storage	2 GB free for the corpus, run artefacts and dependencies
Operating system	Windows 10/11, Linux or macOS with a Python 3.11+ runtime
Runtime and libraries	Python with FastAPI, pandas, NumPy, scikit-learn and pymongo; Node.js with React and Vite
Database and tooling	MongoDB Community Server with JSON fallback; optional JDK for the PySpark engine

Table 3.2 lists the hardware and software the implementation requires at run time.

3.4 Use-Case Diagram and Description

Figure 3.1 shows the actors and use cases. The researcher drives every use case; the corpus and the persistence layer are supporting actors. The use cases drawn in green – inspecting MapReduce statistics, querying the MongoDB run documents and reading the visualization dashboards – are the core portion of this project.

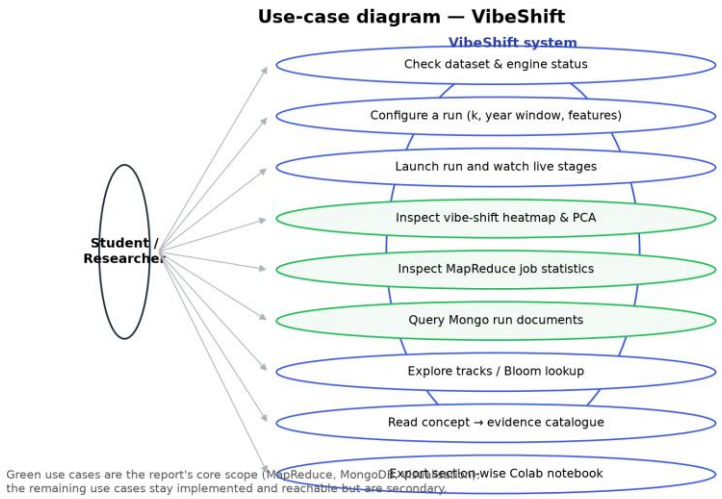


Figure 3.1: Use-case diagram of VibeShift

The primary use case is the scenario exercised whenever the application answers an analytical question. It is triggered by submitting a run configuration; its main flow shows the corpus profile, accepts k and the year window, streams the fifteen stage updates and renders the heatmaps, the PCA map and the analytics panels. If a stage fails, the run is marked failed with its traceback while the remaining analyses still complete, and the post-condition is that the run document, the Parquet track store and the exported notebook are persisted.

Chapter 4: Analysis Modeling

4.1 Class Diagram and Activity Diagram

Figure 4.1 shows the static structure of the backend. The FastAPI routers translate HTTP requests into calls on two collaborating components: the runner, which owns the worker pool and the run documents, and the engine, which performs the numerical work. The analytics suite is a third component that consumes the labelled track frame, so no analysis depends on the internals of the clustering implementation, and the store isolates persistence, which is why the application still works with a JSON fallback when MongoDB is unavailable. In time, a run follows the same decomposition: ingestion, cleaning, vector assembly, clustering, projection and aggregation execute in sequence, then the eight analyses branch from the labelled frame and write their evidence into the same run document; Section 6.1 records the measured cost of each stage.

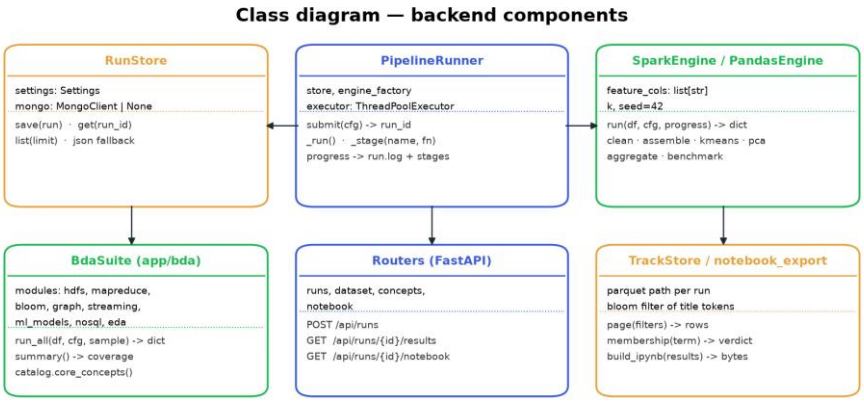


Figure 4.1: Class diagram of the backend components

4.2 Functional Modelling

The data-flow view in Figure 4.2 shows the same system at two levels. At level 0 the researcher supplies a run configuration and receives heatmaps, metrics and concept evidence, while the corpus supplies audio features. At level 1 the processing is decomposed into ingestion and cleaning, vector assembly and clustering, decade aggregation and the analytics suite; data stores D1 (the corpus), D2 (the run documents) and D3 (the per-run Parquet track store) hold the artefacts that flow between them.

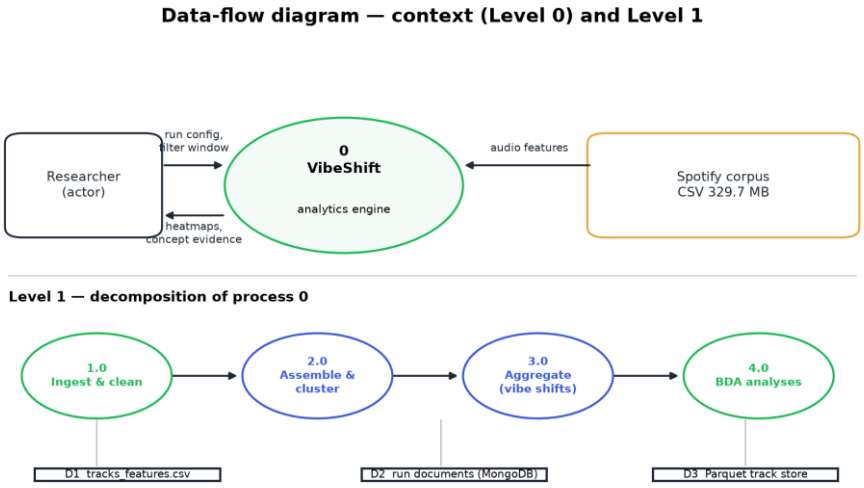


Figure 4.2: Data-flow diagram: context (level 0) and decomposition (level 1)

Chapter 5: Design

5.1 Architectural Design

VibeShift is designed as a three-tier application, shown in Figure 5.1. The presentation tier holds no analytical logic: it renders what the API reports and polls run status while a job is in flight. The application tier owns the pipeline, the engines and the analytics suite, exposes a small REST surface and keeps long-running work off the request path. The data tier separates three artefact types by access pattern – the immutable corpus, the schemaless run documents and the columnar per-run track store – so the track explorer can filter a million rows without reading run metadata. Three decisions deserve justification: the engine abstraction makes the same configuration runnable with or without a JVM; stages write structured counters rather than log text, which is what lets the concepts page quote a live metric; and the analytics stages fail in isolation, so an unavailable MongoDB degrades one panel instead of failing the run.

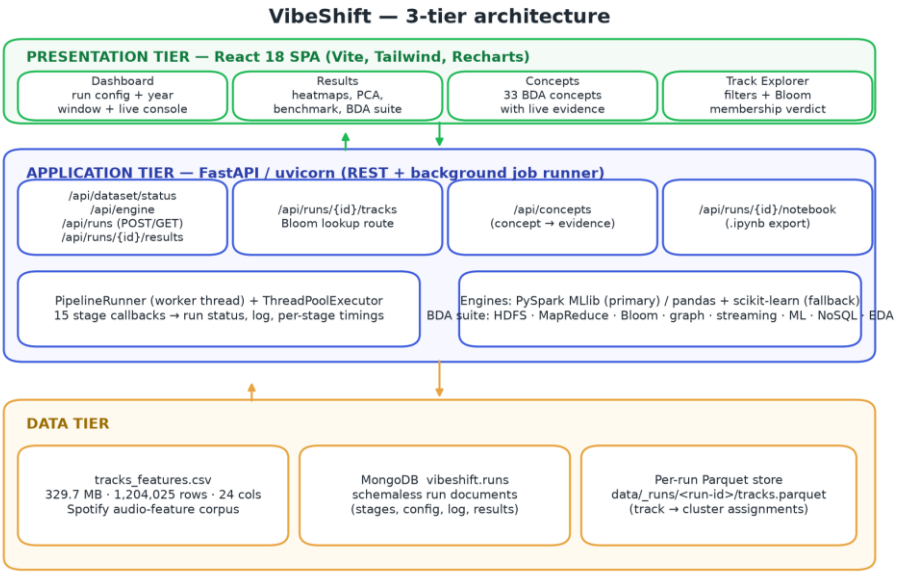


Figure 5.1: Overall system architecture of VibeShift

Table 5.1 summarises the technology chosen for each layer.

Table 5.1: Technology stack

Layer	Technology	Role in the system
Front end	React 18, Vite, Tailwind CSS, Recharts	Dashboard, results, concepts and explorer pages
API and execution	FastAPI on uvicorn [15], worker thread pool	REST endpoints, asynchronous stage execution and status
Engines	PySpark MLlib (primary), pandas and scikit-learn [14] (fallback)	Vector assembly, standardisation, K-Means and PCA
Analytics suite	In-project MapReduce, Bloom, graph, streaming and ML modules	Laboratory-manual techniques applied to the real records

Persistence	MongoDB with JSON fallback, Parquet	Run documents, track store and notebook export

5.2 User Interface Design

The interface is organised around the lifecycle of an experiment rather than around the code structure. Figure 5.2 shows the dashboard: the corpus card reports the real profile of the dataset, including the year bounds the configuration inputs inherit, the engine card explains which engine will execute the run, and the configuration card exposes k, the year window, feature toggles and sampling; panel (d) shows the live console and the run history, so a completed experiment can be reopened instead of repeated.

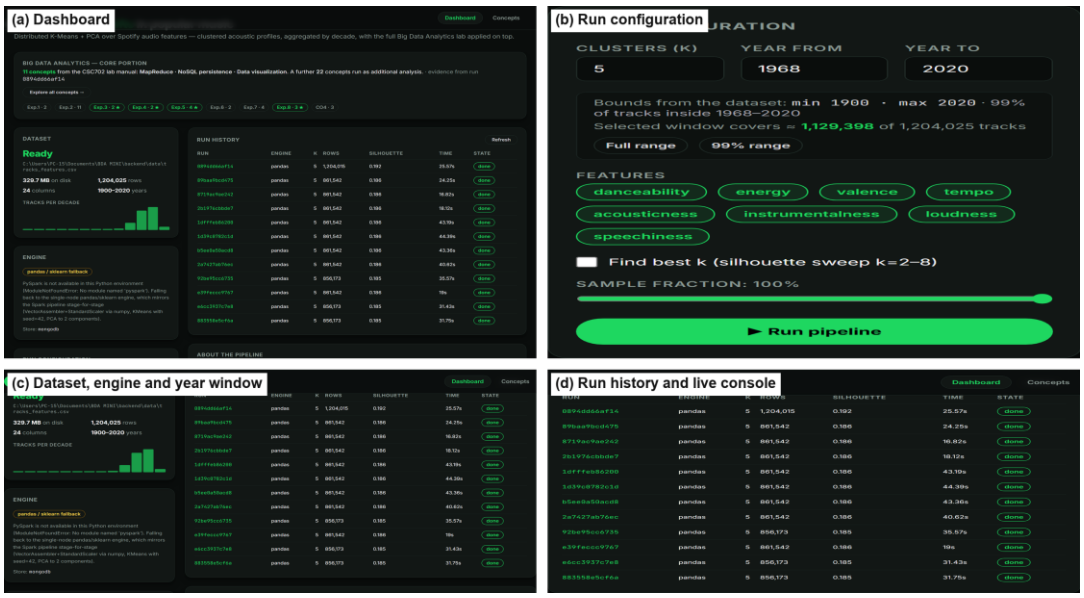


Figure 5.2: Dashboard: corpus profile, run configuration, live console and run history

Figure 5.3 shows the results page, the analytical centre of the application. The decade × cluster heatmap in panel (b) is the primary visualization: each row is a release decade, each column a cluster and each cell the percentage of that decade's tracks the cluster contains. Panel (c) documents what each cluster means through its mean feature profile and size, and panel (d) shows the PCA projection and the per-decade feature trends that corroborate the heatmap.



Figure 5.3: Results page: headline metrics, vibe-shift heatmap, cluster profiles and projection

Chapter 6: Implementation

6.1 Working of the Project

A run begins by reading the corpus. The recorded run ingested 1,204,025 rows from a 330 MB CSV file and retained 1,204,015 rows after cleaning. Cleaning performs the fixes the corpus demands: years are cast through a float so values such as 2004.0 parse, non-positive years are dropped, features are coerced to numeric with malformed values becoming missing, rows with missing features are removed, duplicate track identifiers are collapsed and a decade column is derived. Vector assembly standardises the eight features with the population standard deviation to match Spark's StandardScaler, so both engines cluster identical matrices. Table 6.1 describes the corpus and Table 6.2 records the measured cost of each stage with the headline metrics of the run.

Table 6.1: Description of the corpus used for the recorded run

Property	**Value**
Source	Public Spotify "1.2M+ Songs" dataset (tracks_features.csv)
Size and shape	329.7 MB, 1,204,025 rows, 24 columns
Release years	1900–2020 for valid rows (10 invalid years excluded); 99% of tracks fall in 1968–2020
Clustering features	danceability, energy, valence, tempo, acousticness, instrumentalness, loudness, speechiness
Analysed rows	1,204,025 ingested, 1,204,015 clean

Table 6.2: Measured pipeline cost and headline metrics for the recorded run

Stage	**Measure**	**Value**
ingest	rows read from the CSV file	1,204,025
clean	rows retained after cleaning	1,204,015
assemble	standardised matrix assembly	0.27 s
k-means	fit time for k = 5	4.68 s
pca and aggregate	projection and decade × cluster aggregation	0.06 s and 0.47 s
benchmark	vectorised versus naive loop aggregation	12.8× faster
analytics suite	eight analyses	10.2 s, 8 of 8 successful
whole run	runner wall time	25.6 s
quality	silhouette and inertia for k = 5	0.192 and 4,639,125

6.2 MapReduce – Core Portion (Experiments 4 and 5)

The MapReduce implementation is an in-process runtime with the phases a distributed job would exhibit: each map task emits key-value pairs, a combiner aggregates them locally, pairs are partitioned by crc32(key) modulo the reducer count, and each reducer receives one partition. Every phase is timed and counted, which is what makes the inspector in Figure 6.3(a) an inspection rather than a description. Figure 6.1 shows the model with the counters recorded for the word-count job. The word-count job over track titles emitted 69,298 map output pairs and the combiner cut them to 19,609, removing 71.7% of the shuffle volume; the per-decade aggregate job absorbed 99.95% of its pairs. Five jobs run over a 25,000-row sample on every run, so the runtime stays bounded while the statistics remain real. The same framework implements the map-side join that attaches cluster labels, a top-N ranking by energy and the inverted index that backs the search box.

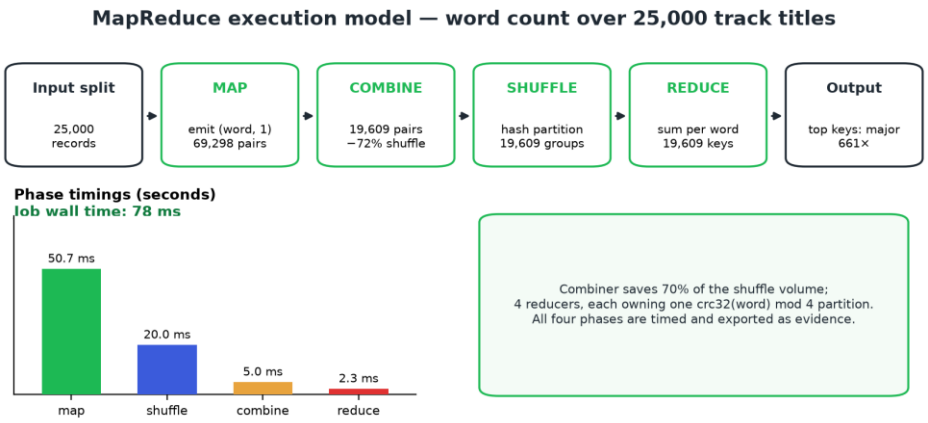


Figure 6.1: MapReduce execution model with the counters recorded for the word-count job

6.3 NoSQL Persistence with MongoDB – Core Portion (Experiment 3)

Every run is persisted as one document in the vibeshift.runs collection, which held 20 documents of 2,418 bytes on average in the recorded run (backend mongodb, connected = true). The document model is a deliberate NoSQL decision: a run has a nested array of stage records whose length grows as the pipeline grows, an embedded configuration whose shape differs between runs with and without the best-k sweep, and a log array, none of which maps cleanly onto fixed relational columns. The console in Figure 6.3(b) reflects over the collection to enumerate the field paths and their types, reports live collection statistics, shows a sample document and lists the queries and aggregation stages the application issues. It is

read-only, so an inspection cannot mutate the store, and when MongoDB is unavailable the same panel is served from the JSON fallback.

6.4 Statistical Computing and Visualization – Core Portion (Experiment 8)

The statistics module profiles all eight features over the 1,204,015 clean rows – count, mean, standard deviation, quartiles, skewness and kurtosis – and computes the full Pearson correlation matrix. The strongest relationships are energy–loudness (r = 0.82), energy–acousticness (r = -0.80) and acousticness–loudness (r = -0.67); the last two are two views of the same signal, and together they explain why the acoustic and high-energy clusters separate so cleanly in feature space. The module also reports outlier counts on a robust z-score criterion and re-scores the features by variance contribution.

Visualization is where the analyses meet the clustering results. Figure 6.2(a) reproduces the correlation heatmap rendered from those statistics and Figure 6.2(b) plots the cluster sizes beside the stacked decade composition, which is the heatmap of Figure 5.3(b) expressed as a distribution. Together they show the finding the project was built to measure: the acoustic and instrumental clusters dominate the earliest decades, their share falls after the 1970s, and the high-energy electronic cluster grows from a marginal presence in the 1960s to the largest single share of the 2010s.

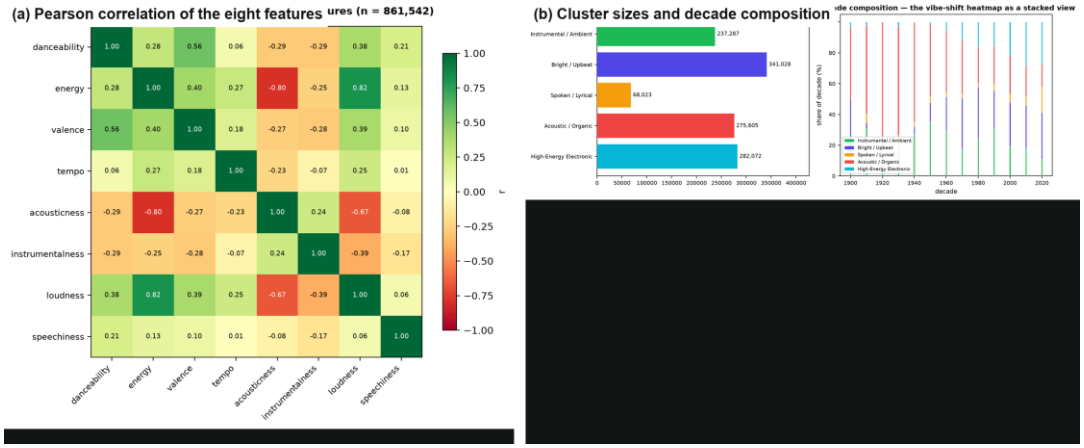


Figure 6.2: Statistics and visualization: correlation matrix and cluster/decade composition

6.5 Additional Big Data Analytics Analyses

The remaining techniques of the manual are executed on every run and reported as supporting evidence. The HDFS analysis models the corpus as a replicated file on a four-node, two-rack cluster and reports the block map and DataNode occupancy. The Bloom filter indexes the artist vocabulary with a 383,403-bit array and 7 hash functions; probing it with 2,000 terms produced no false negatives, which is what makes it safe for the membership verdict in the

track explorer. The graph analysis builds a k-nearest-neighbour similarity graph and detects communities, reporting degree, modularity and PageRank centralities. The streaming analysis replays the corpus as an event-time stream in 60 micro-batches with a sliding window and detects 17 drift points, a quantitative statement about when the corpus stops being stationary. The supervised analysis frames “is this track modern?” as a classification task and selects between three models by F1 and ROC AUC, and retains RandomForest as the best with an accuracy of 0.819. Table 6.3 lists the analyses with the metric each produced, and Figure 6.3 shows their interface panels.

Table 6.3: Big Data Analytics analyses executed in the recorded run

Analysis	Exp.	Technique	Evidence produced
HDFS and Hadoop	1	Block splitting, 3× replication, rack-aware placement	3 blocks, 329.7M
MapReduce	4 and 5	map, partition, combine, shuffle, reduce	5 jobs, 226,286 map output pairs
Bloom filter	6	Bit array with k hash functions, tunable false-positive rate	m = 383,403, k = 7, empirical FPR 0.0075, no false negatives
Graph mining	7	k-NN similarity graph, communities, PageRank	200 nodes, 557 edges, modularity 0.677
Stream processing	CO4	Event-time micro-batches, sliding window, drift detection	60 batches, 17 drift points
Supervised learning	2	Train/test split, three models, F1 and ROC AUC	best RandomForest, accuracy 0.819
NoSQL (MongoDB)	3	Schemaless documents, reflection, queries, aggregation	20 documents in vibeshift.runs
Statistics and visualization	8	Descriptive statistics, correlation, charts	8 feature profiles, correlation matrix over 1,204,015 rows

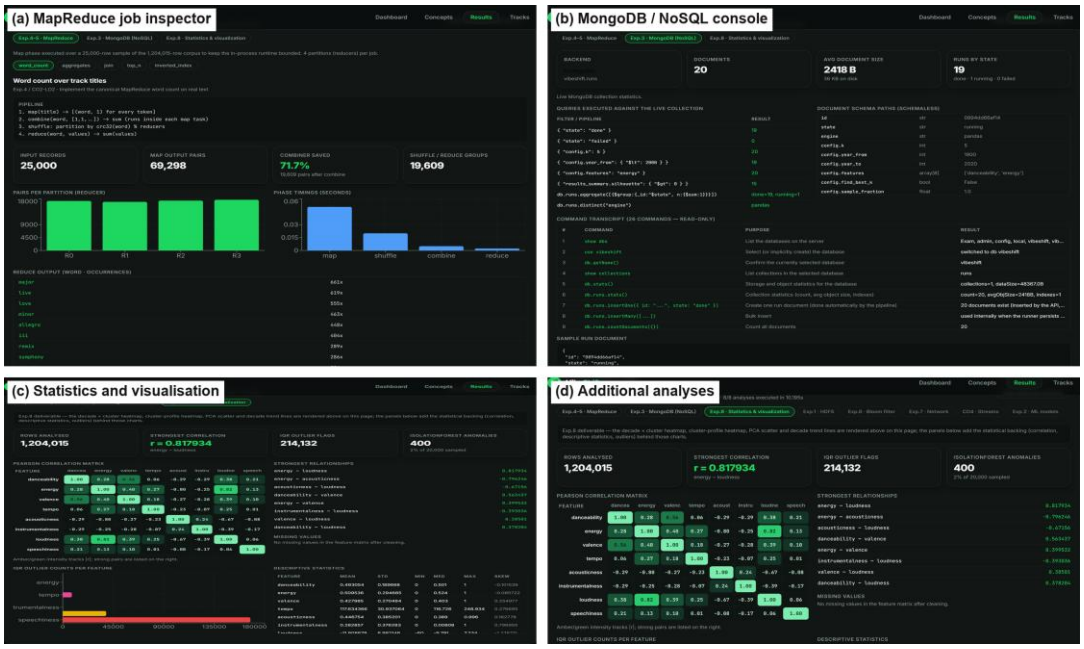


Figure 6.3: Big Data Analytics suite: MapReduce inspector, NoSQL console, statistics panel and additional analyses

6.6 Concept Catalogue and Track Explorer

The interface also makes the coverage of the manual explicit. The catalogue holds 33 concepts across 9 experiment groups; 33 carry a live metric from the selected run and 11 are

marked as the core portion of this project. Each concept is listed with its experiment, what it means, how it is implemented and the number it produced, followed by the JSON path holding the evidence so a reviewer can verify it. The track explorer filters the per-track cluster assignments by cluster or decade and searches by title; the search box issues a Bloom-filter membership query on the server and prints the verdict with the probability of a false positive, connecting the probabilistic data structure to a visible feature.

Chapter 7: Conclusion

VibeShift converts an exploratory Colab notebook into a reproducible analytics system. A single run over 1,204,025 tracks is executed by fifteen instrumented stages, produces five acoustic clusters with a silhouette score of 0.192, and expresses the change in popular music as a decade × cluster distribution; the recorded run completed in 25.6 seconds on one machine. Each cluster is named from its own measured profile, so the labels in the interface are derived rather than assumed, and the pipeline work – reading, cleaning, standardising, clustering, projecting, aggregating and benchmarking – is separable from the analytics that follow it.

The core portion is MapReduce, NoSQL persistence and statistical visualization. MapReduce is a runtime whose phases are counted and timed, so the combiner's effect on shuffle volume is measurable rather than assumed; MongoDB stores each run as a schemaless document and the application demonstrates the document model, reflection, queries and the aggregation pipeline on live data while staying read-only; and statistical profiling with visualization carries the result to the reader, where the correlation matrix, the vibe-shift heatmap and the stacked decade composition together show the acoustic-to-electronic transition. The five remaining analyses run on every execution and are exposed as additional evidence.

The limitations define the next steps. The pipeline runs on a single node, so the distributed engine is only used when a JVM is available, and deploying the same configuration on a Spark cluster would make the shuffle and partitioning behaviour directly comparable with the in-process model. The MapReduce, graph and streaming analyses work on bounded samples whose sizes are reported rather than hidden. Replacing the stream replay with a live broker would turn drift detection into an operational signal, learned track embeddings in place of the eight curated features would let the clustering discover similarity the feature set cannot express, and authentication with per-user history would turn the application into a shared tool rather than a local one.

REFERENCES

[1] J. Dean and S. Ghemawat, “MapReduce: Simplified Data Processing on Large Clusters,” Proceedings of the 6th Symposium on Operating Systems Design and Implementation (OSDI), 2004.

[2] K. Shvachko, H. Kuang, S. Radia and R. Chansler, “The Hadoop Distributed File System,” Proceedings of the 26th IEEE Symposium on Mass Storage Systems and Technologies (MSST), 2010.

[3] M. Zaharia et al., “Apache Spark: A Unified Engine for Big Data Processing,” Communications of the ACM, vol. 59, no. 11, 2016.

[4] J. MacQueen, “Some Methods for Classification and Analysis of Multivariate Observations,” Proceedings of the 5th Berkeley Symposium on Mathematical Statistics and Probability, 1967.

[5] T. Hastie, R. Tibshirani and J. Friedman, The Elements of Statistical Learning: Data Mining, Inference and Prediction, 2nd ed., Springer, 2009.

[6] P. J. Rousseeuw, “Silhouettes: A Graphical Aid to the Interpretation and Validation of Cluster Analysis,” Journal of Computational and Applied Mathematics, vol. 20, 1987.

[7] M. E. J. Newman, “Modularity and Community Structure in Networks,” Proceedings of the National Academy of Sciences, vol. 103, no. 23, 2006.

[8] L. Page, S. Brin, R. Motwani and T. Winograd, “The PageRank Citation Ranking: Bringing Order to the Web,” Stanford InfoLab Technical Report, 1999.

[9] B. H. Bloom, “Space/Time Trade-offs in Hash Coding with Allowable Errors,” Communications of the ACM, vol. 13, no. 7, 1970.

[10] T. Akidau et al., “The Dataflow Model: A Practical Approach to Balancing Correctness, Latency and Cost in Massive-Scale, Unbounded, Out-of-Order Data Processing,” Proceedings of the VLDB Endowment, vol. 8, no. 12, 2015.

[11] MongoDB Inc., MongoDB Manual: Documents, Queries and Aggregation Pipeline, 2024. [Online]. Available: https://www.mongodb.com/docs/manual/

[12] M. Kleppmann, Designing Data-Intensive Applications, O'Reilly Media, 2017.

[13] Spotify, “Audio Features and Track Analysis,” Spotify for Developers documentation, 2024. [Online]. Available: https://developer.spotify.com/documentation/web-api/reference/

[14] F. Pedregosa et al., “Scikit-learn: Machine Learning in Python,” Journal of Machine Learning Research, vol. 12, 2011.

[15] S. Ramírez, “FastAPI documentation,” 2024. [Online]. Available: https://fastapi.tiangolo.com/

ACKNOWLEDGEMENTS

We thank our guide for the guidance and feedback that shaped this project and the Department of Computer Engineering for the laboratory and computing facilities. We also acknowledge the maintainers of the public Spotify corpus and of the open-source libraries used here – pandas, scikit-learn, PySpark, FastAPI, React and MongoDB – whose documentation made the implementation practical, and our classmates who reviewed early versions of the interface.